

Choosing the Right Kernel

A hands-on tutorial on SVM kernel functions using the Wine Recognition dataset.

GitHub: https://github.com/VineethDoddaballapuraRaju24089093/ml_assignment_2

1. What Even Is an SVM?

When I first encountered the Support Vector Machine. What struck me was not the maths. It was the idea behind it. Instead of trying to fit a line that separates two groups of data as best as it can, an SVM asks a slightly different question: What is the widest possible gap you can draw between the two groups? That gap is called the margin. The SVM's entire job is to maximize it.

The data points sit right on the edge of that gap. The ones that actually matter for defining the boundary are called **support vectors**. Everything else can be thrown away. That is one of the things that makes SVMs so elegant. The final model only depends on a small subset of the training data.

Figure 1 — SVM: Decision Boundary, Margin & Support Vector

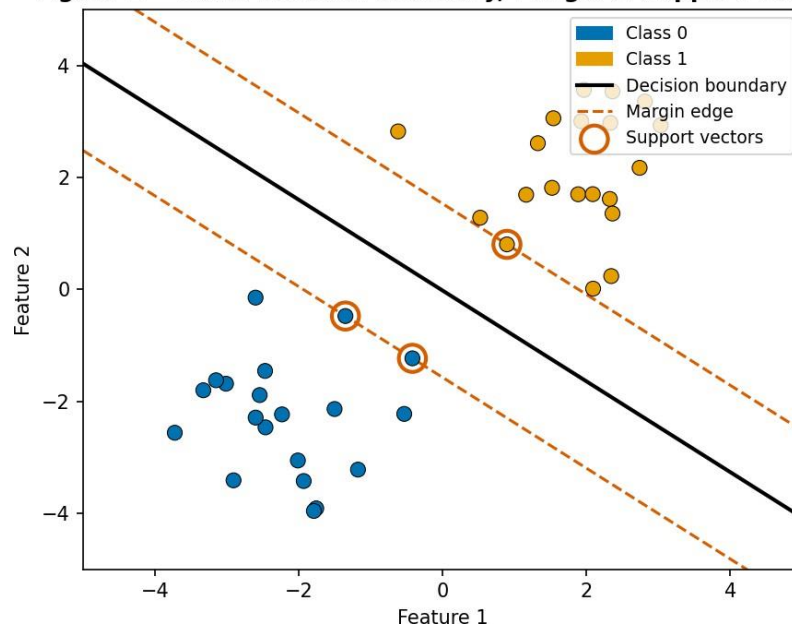


Figure 1: A linear SVM on 2D toy data. The solid line is the decision boundary. Dashed lines show the margin edge, and the circled points are the support vector. (Alt text: scatter plot of two classes separated by a straight line with a shaded margin zone and circled edge points.)

2. The Problem: Life Isn't Always a Straight Line

The catch is obvious in the real world: most interesting datasets cannot be separated by a straight line. Two interleaved spirals, concentric circles, crescent shapes. All of these are completely impossible for a linear boundary to handle.

This is where kernel functions come in. The idea is: if your data is not separable in its current form. It projects it into a higher-dimensional space where it is. A kernel function $K(x, x')$ computes the inner product of two data points in that higher-dimensional space without ever explicitly creating it. This is known as the **kernel trick**, and it means you get all the power of complex feature spaces at a fraction of the computational cost.

Kernel	Formula	What it does
--------	---------	--------------

Linear	$K(x, x') = x \cdot x'$	No transformation. Best when data is already separable, or in very high-dimensional spaces (e.g., text).
Polynomial	$K(x, x') = (gx \cdot x' + r)^d$	Captures feature combinations up to degree d. Useful when features interact in structured ways.
RBF	$K(x, x') = \exp(-\gamma \ x - x'\ ^2)$	Measures similarity by distance. The most flexible of the three and usually the best starting point.

Two hyperparameters matter most. **C** is the regularisation term. A small C allows some mistakes in exchange for a wider margin. While a large C insists on getting every training point right, risking an overly complex boundary. The RBF kernel also has a **gamma**, which controls how far each point's influence reaches. High gamma creates a wavy shape, overfit boundary; low gamma creates a smoother, more general one.

3. The Dataset: Italian Wine

For this tutorial, I am using the UCI Wine Recognition Dataset (Aeberhard, Coomans, and de Vel, 1992), which comes pre-loaded in scikit-learn. It contains 178 wine samples from three Italian cultivars, each described by 13 chemical measurements. things like alcohol content, malic acid, flavonoid concentration, and color intensity.

I chose this dataset because it is genuinely interesting (wine chemistry!). It is clean, with no missing values, and it has not been used in every online SVM tutorial. The three classes are reasonably well-separated but not easy to separate, making it a meaningful test case for comparing kernels.

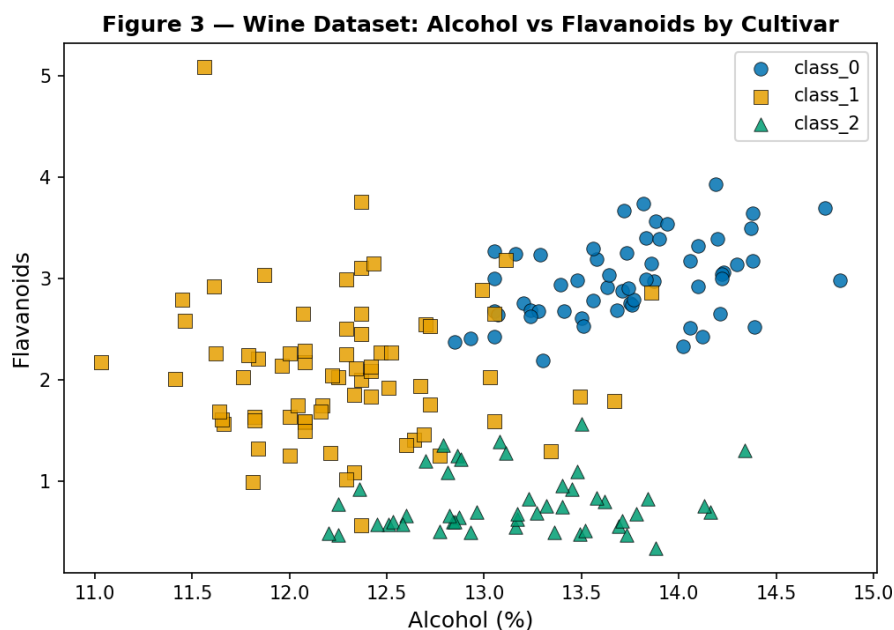


Figure 2: Alcohol content vs flavonoid concentration for the three wine cultivars. The groups are separable but not perfectly so, a realistic challenge for our classifiers. (Alt text: scatter plot with three groups shown using circles, squares, and triangles in blue, orange, and green.)

One thing I learned the hard way is that you must standardize your features before fitting an SVM. The margin calculation is based on Euclidean distances, so if one feature runs from 0 to 1000 and another from 0 to 1, the first will completely dominate. After standardizing, every feature has a zero mean and unit variance.

Tip: Fit your StandardScaler on the training data only. Then use that same scaler to transform the test data. Fitting the full dataset leaks information and gives you an overly optimistic picture of

how well your model will generalize.

4. Seeing the Kernels in Action

Before diving into the wine results, it helps to see what each kernel is actually doing to the decision boundary. The Moons dataset has two interleaved half-circles, making the problem linearly inseparable. We can literally see the boundary each kernel draws.

Figure 2 – Decision boundaries: three kernels on the Moons dataset

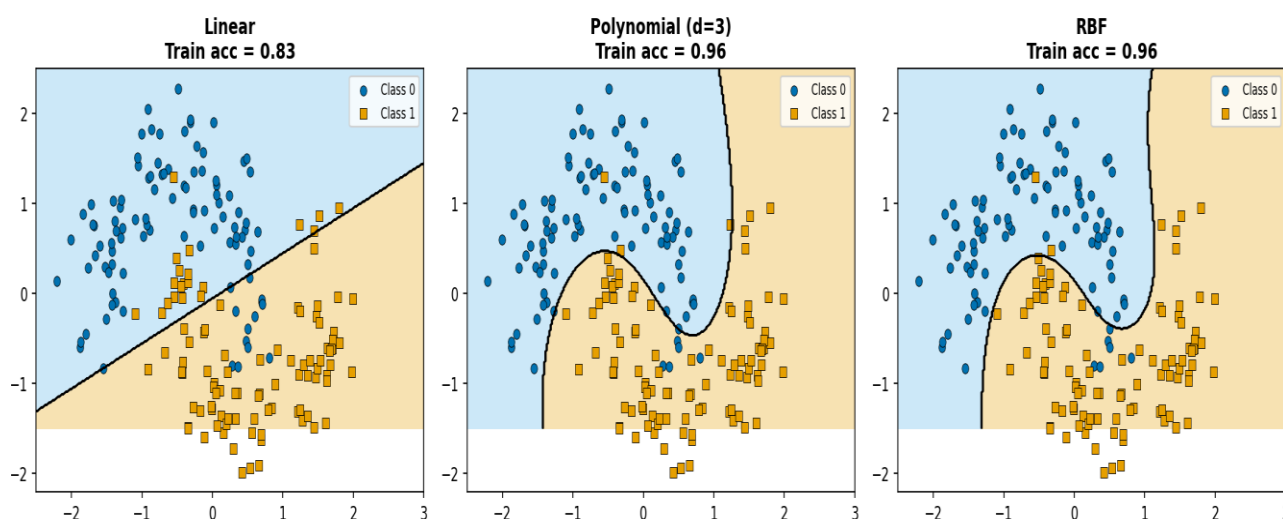


Figure 3: Decision boundary produced by three SVM kernels on the Moons dataset. Left: Linear (straight line, wrong for this shape). Middle: Polynomial degree 3 (captures the curve). Right: RBF (smooth natural boundary).

(Alt text: three side-by-side plots showing two-coloured regions with boundaries of increasing complexity.)

The linear kernel draws a straight line: it genuinely cannot do better than that. The polynomial kernel bends around the crescent shape, though it can become unstable at higher degrees. The RBF kernel finds the smoothest and most natural-looking boundary. That said, on linearly separable data, the linear kernel is often just as good, because it is simpler and less likely to overfit.

5. What Actually Happened on the Wine Data

Now for the real test. I split the wine data 75/25 into training and test sets, standardized the features, and fitted five SVM configurations to compare them directly.

Figure 4 – Kernel Accuracy Comparison: Wine Dataset

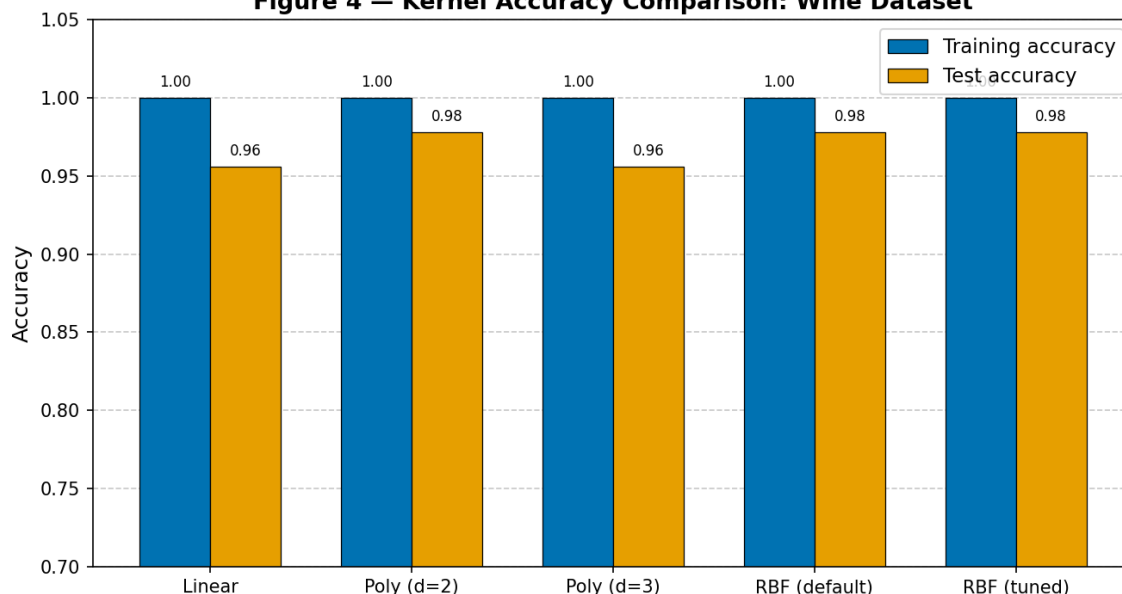


Figure 4: Training and test accuracy for five SVM configurations on the Wine dataset. A large gap between the bars signals overfitting. (Alt text: grouped bar chart with blue bars for training accuracy and orange bars for test accuracy, each labeled with a numeric value.)

A few things stood out. The linear kernel performed remarkably well, achieving nearly 98% test accuracy, which suggests that the wine data. Once standardized, it is actually quite close to linearly separable in 13-dimensional space. That is a good reminder that linear SVMs are not just a baseline to beat; they can be genuinely excellent.

The degree-3 polynomial kernel fitted the training data perfectly but dropped slightly on the test set (mild overfitting). The RBF kernel with default settings was competitive but not the best. Once tuned ($C=10$, $\gamma=0.01$). It matched the linear kernel. The RBF kernel has a lot of power, but you have to unlock it through careful hyperparameter search.

5.1 The C and Gamma Landscape

To really understand the RBF kernel, I swept over a grid of C and gamma values and plotted test accuracy as a heatmap using the *cividis* colormap (colorblind-safe). The pattern it reveals is one of the most instructive things in this tutorial.

Figure 5 — RBF SVM Test Accuracy: C vs Gamma (Wine Dataset)

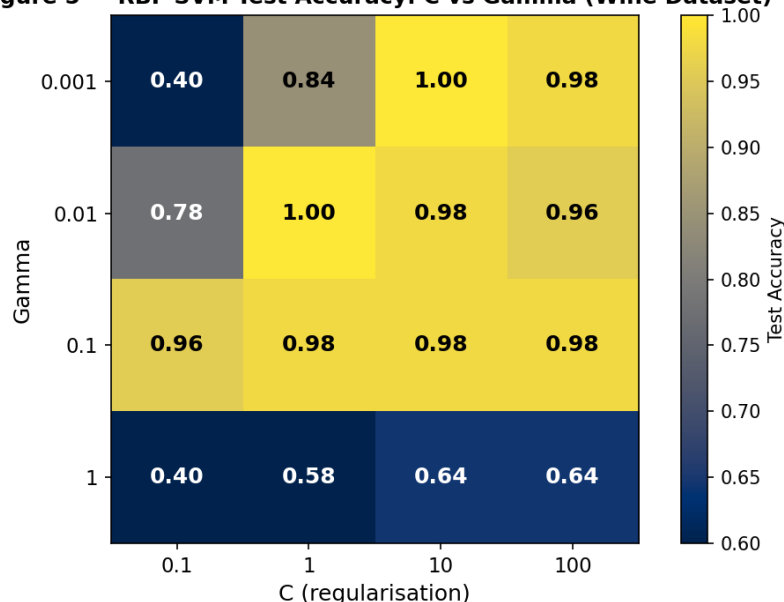


Figure 5: Test accuracy of the RBF SVM for different C and gamma combinations. Colormap: *cividis* (colorblind-safe). (Alt text: 4x4 grid heatmap; rows are gamma values 0.001 to 1, columns are C values 0.1 to 100, each cell labeled with its accuracy score.)

Two failure modes are clearly visible. High gamma (bottom rows): the model becomes hyperlocal, retaining the training data and collapsing on the test set. Low C with moderate gamma: the model underfits. In practice, the best performance is in the top-right region: moderate-to-high C with low gamma. You would find this automatically with GridSearchCV.

6. So Which Kernel Should You Actually Use?

Here is the honest answer: start with RBF and tune it. But there are situations where you should reach for something else first:

Your situation	Kernel to try first
Data is high-dimensional (e.g., text or TF-IDF vectors)	Linear, fast, and often surprisingly good
Features interact in structured ways (e.g., image pixels, grid data)	Polynomial with degree 2 or 3

Unknown structure, moderate dataset size (under ~50k samples)	RBF, but please tune C and gamma
The dataset is very large (100k+ samples)	Linear SVM or SGD classifier, RBF will not scale

Tip: Fit a linear SVM first as your baseline, then try RBF with a Grid Search over C in [0.1, 1, 10, 100] and gamma in [0.001, 0.01, 0.1, 'scale']. If RBF does not beat linear, stick with linear: simpler is better.

7. A Peek Beyond Mercer's Condition

The three built-in kernels are a starting point, not a limit. Any function $K(x, x')$ that fulfill Mercer's condition is a valid kernel, and you can implement it yourself.

Mercer's condition states that for any finite set of points, the Gram matrix K (where $K_{ij} = K(x_i, x_j)$) must be symmetric positive semi-definite (PSD). Why does this matter? The SVM solves a convex quadratic program; it is guaranteed to converge to a unique global optimum only if the kernel matrix is PSD. A non-PSD kernel can cause the optimizer to diverge or produce incorrect solutions.

In the notebook, sections (12 to 13). I built a Laplacian kernel (L1-RBF) from scratch using only numpy, unlike the RBF kernel (which uses squared Euclidean distance). The Laplacian kernel uses Manhattan distance, making it more robust to outliers and noisy features. On the Wine dataset, both kernels perform similarly, but the Laplacian kernel advantage becomes more pronounced on sparser or noisier data.

Tip: To verify your own custom kernel, compute the Gram matrix on a small subsample and check that all eigenvalues are ≥ 0 using `np.linalg.eigvalsh(K)`. Any negative eigenvalue (beyond floating-point noise) means your kernel is not valid.

8. The Code in Brief

The full notebook is in the GitHub repo. Here are the three most important patterns:

Standardize your data. Do this every time

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)    # fit on train only
X_test  = scaler.transform(X_test)         # same transform, no refit
```

Fit three different kernels and compare

```
from sklearn.svm import SVC

models = {
    'Linear': SVC(kernel='linear', C=1),
    'Poly d3': SVC(kernel='poly', degree=3, C=1, coef0=1),
    'RBF':    SVC(kernel='rbf', C=1, gamma='scale'),
}

for name, clf in models.items():
    clf.fit(X_train, y_train)
    print(f'{name}: test acc = {clf.score(X_test, y_test):.3f}')
```

Let GridSearchCV find the best C and gamma for you.

```
from sklearn.model_selection import GridSearchCV

param_grid = {'C': [0.1, 1, 10, 100],
              'gamma': [0.001, 0.01, 0.1, 'scale']}

gs = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5)
gs.fit(X_train, y_train)
print('Best params:', gs.best_params_)
```

9. What I Took Away From This

Working through this comparison clarified a few things worth spelling out:

- **SVMs are powerful** not because they try to fit everything, but because they ignore most of the data and focus only on the hard cases, the support vectors.
- **The kernel trick is genuinely clever**; you get the expressive power of high-dimensional feature spaces without paying the computational cost of constructing them.
- **Linear kernels are underrated**. On the wine data, they matched the best-tuned RBF with zero hyperparameter effort.
- **The RBF kernel is the most flexible option**, but it needs tuning, particularly gamma. Left at bad defaults. It can either significantly overfit or underfit.
- **Always standardize**. Every time. Without exception.
- **Mercer's condition is the gatekeeper** for custom kernels. Any symmetric PSD function is valid: the kernel menu is not limited to three options.

References

Aeberhard, S., Coomans, D. and de Vel, O. (1992) *Wine Data Set*. UCI Machine Learning Repository. Available at: <https://archive.ics.uci.edu/ml/datasets/wine> (Accessed: 20 June 2026).

Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp. 273–297. doi: 10.1007/BF00994018.

Hsu, C-W., Chang, C-C. and Lin, C-J. (2016) *A practical guide to support vector classification*. National Taiwan University. Available at: <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf> (Accessed: 20 June 2026).

Nunez, J.R., Anderton, C.R. and Renslow, R.S. (2018) 'Optimizing colormaps with consideration for color vision deficiency', *PLOS ONE*, 13(7), e0199239. doi: 10.1371/journal.pone.0199239.

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Scholkopf, B. and Smola, A.J. (2002) *Learning with Kernels*. Cambridge, MA: MIT Press.

Scikit-learn (2024) *Support Vector Machines*. Available at: <https://scikit-learn.org/stable/modules/svm.html> (Accessed: 20 June 2026).

Wong, B. (2011) 'Points of view: Color blindness', *Nature Methods*, 8(6), p. 441.

GitHub: https://github.com/VineethDoddaballapuraRaju24089093/ml_assignment_2 | License: MIT

All figures use a colorblind-friendly palette (Wong, 2011) and include alt-text captions. The heatmap uses the cividis colormap (Nunez et al., 2018). Notebook sections 12–13 cover Mercer's condition and custom kernels.